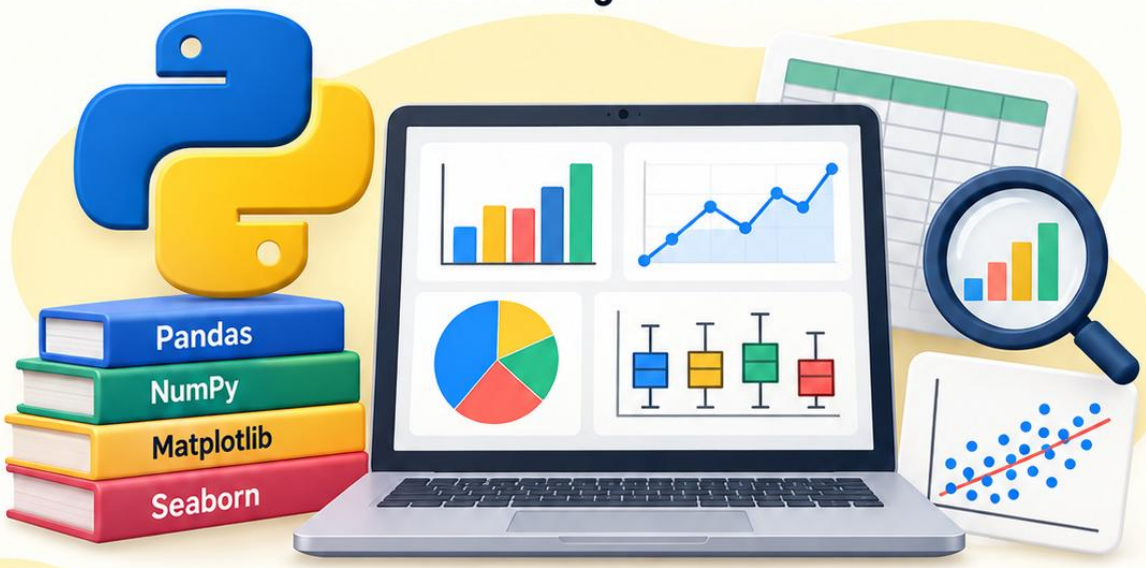
EDA (Exploratory Data Analysis) Notes

Complete Guide to Understand, Explore and Extract Insights from Data



DataXOdyssey

Exploratory Data Analysis (EDA) Fundamentals

What is EDA?

Exploratory Data Analysis (EDA) is the process of understanding a dataset, finding patterns, spotting anomalies and generating insights before building models or making business decisions.

Why is EDA Important?

- Helps understand data and business context
- Identifies data quality issues early
- Reveals patterns, trends and relationships
- Guides feature engineering and model building
- Helps make better data-driven decisions

Objectives of EDA

- [x] Understand the structure of the data
- [x] Check data quality (missing values, duplicates, etc.)
- [x] Summarize key statistics
- [x] Explore distributions and identify outliers
- [x] Find patterns, trends and relationships
- [x] Generate initial hypotheses and insights

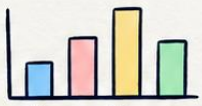
Types of EDA

- **Univariate Analysis**
  Analyze one variable at a time
- **Bivariate Analysis**
  Study the relationship between two variables
- **Multivariate Analysis**
  Analyze multiple variables together

Typical EDA Workflow

1. Understand the problem and data
2. Explore data structure and data types
3. Check data quality
4. Perform descriptive analysis
5. Analyze distributions, relationships and outliers
6. Draw insights and document findings

Outcome of EDA



Key patterns and trends



Data quality issues identified



Meaningful insights and hypotheses



Basis for data modeling and business decisions

DataXOdyssey

Data Structure & Initial Inspection

1. Understand the Dataset

Start EDA by understanding what data you have – its size, structure, columns, data types and a quick look at the actual values.

2. Load and View the Data

```
import pandas as pd
df = pd.read_csv('data.csv')
```

```
df.head()      # first 5 rows
df.tail()      # last 5 rows
df.sample(5)   # random 5 rows
```

df.head()

ID	Name	Age	City
1	A	25	Delhi
2	B	30	Mumbai
3	C	28	Pune
4	D	32	Chennai
5	E	27	Kolkata

3. Dataset Size (Shape)

```
df.shape   # (rows, columns)
```

Output:

(1000, 12)

This means:
1000 rows
12 columns

4. Column Names

```
df.columns
```

Output:

```
Index(['ID', 'Name', 'Age',
       'City', 'Salary', ...],
      dtype='object')
```

5. Data Types

```
df.dtypes
```

Column	Data Type
ID	int64
Name	object
Age	int64
City	object
Salary	float64
Join_Date	datetime64[ns]

6. Quick Summary

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 12 columns):
 #   Column      Non-Null Count  Dtype
 0   ID          1000 non-null   int64
 1   Name        1000 non-null   object
 2   Age         1000 non-null   int64
...
 11  Join_Date   980 non-null    datetime64[ns]
dtypes: datetime64[ns](1), int64(3), float64(2),
object(6)
memory usage: 93.9+ KB
```

7. Descriptive Statistics

```
df.describe()
```

	Age	Salary	Experience
count	1000	1000	1000
mean	29.5	65000	5.2
std	4.8	15000	2.9
min	20	30000	0
25%	26	50000	3
50%	29	62000	5
75%	33	78000	7
max	45	120000	15

8. Unique Values

```
df.nunique()
```

Column	Unique Values
ID	1000
Name	980
Age	26
City	8
Salary	450
Join_Date	365

9. Check for Missing Values

```
df.isnull().sum()
```

Column	Missing Values
ID	0
Name	20
Age	0
City	5
Salary	10
Join_Date	30

10. Check for Duplicates

```
df.duplicated().sum()
```

Output:

8

To view duplicate rows:

```
df[df.duplicated()]
```

11. View Sample Values

```
df['City'].value_counts()
```

City	Count
Mumbai	250
Delhi	200
Bangalore	150
Pune	120
Chennai	100
Kolkata	80
Hyderabad	70
Other	30

12. Initial Observations

- ✅ Dataset has 1000 rows and 12 columns
- ✅ Mixed data types (numeric, text, date)
- ✅ Missing values in some columns
- ✅ 8 duplicate rows
- ✅ Shows basic distribution and range of numeric columns
- ✅ Gives a clear idea of what further cleaning and analysis is needed

DataXOdyssey

Data Quality & Missing Values

1. What is Data Quality?

Data quality means data is accurate, complete, consistent and reliable for analysis.

2. Common Data Quality Issues

- Missing values
- Duplicate records
- Inconsistent formats
- Invalid or out-of-range values
- Incorrect data types
- Typos and extra spaces

3. Check Missing Values

```
df.isnull().sum()
```

Column	Missing Values
ID	0
Name	20
Age	0
City	5
Salary	10
Join_Date	30

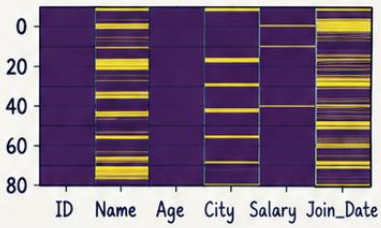
4. Missing Values Percentage

```
(df.isnull().sum() / len(df)) * 100
```

Column	% Missing
ID	0.00
Name	2.00
Age	0.00
City	0.50
Salary	1.00
Join_Date	3.00

5. Visualize Missing Values

```
import seaborn as sns
sns.heatmap(df.isnull(),
            cbar=False,
            cmap='viridis')
```



6. Handle Missing Values

Drop rows with missing values

```
df.dropna()
```

Drop columns with missing values

```
df.dropna(axis=1)
```

Fill with a constant value

```
df['City'].fillna('Unknown')
```

Fill with mean / median (numeric)

```
df['Salary'].fillna(df['Salary'].median())
```

Fill with mode (categorical)

```
df['City'].fillna(df['City'].mode()[0])
```

7. Check for Duplicates

```
df.duplicated().sum()
```

Output: 8 duplicate rows

View duplicate rows:

```
df[df.duplicated()]
```

Remove duplicates:

```
df.drop_duplicates()
```

8. Check for Invalid Values

Examples:

Negative age (should be > 0)

```
df[df['Age'] <= 0]
```

Salary out of expected range

```
df[(df['Salary'] < 10000) |
   (df['Salary'] > 500000)]
```

Invalid date format

```
pd.to_datetime(df['Join_Date'],
    errors='coerce').isnull().sum()
```

Inconsistent categories (e.g. city names)

```
df['City'].str.strip().str.title().unique()
```

Key Takeaways

- ✅ Check missing values and understand the pattern
- ✅ Identify and handle duplicates
- ✅ Look for invalid or out-of-range values
- ✅ Ensure correct data types
- ✅ Standardize text and category values
- ✅ Fix inconsistencies and typos
- ✅ Clean and prepare data before analysis for more accurate insights

Descriptive Statistics

Summarizing and understanding the main characteristics of your data

1. What is Descriptive Statistics?

Descriptive statistics summarize the main features of a dataset using simple metrics, helping us understand the data, identify patterns and prepare for further analysis.

2. Types of Descriptive Statistics

- **Measures of Central Tendency**
  Mean, Median, Mode
  (typical or average value)
- **Measures of Dispersion**
  Range, Variance, Standard Deviation, IQR (spread of the data)
- **Measures of Position**
  Minimum, Maximum, Quartiles, Percentiles
- **Measures of Shape**
  Skewness and Kurtosis
  (distribution shape)

3. Central Tendency

Metric	Formula	Description
Mean	$\frac{\Sigma x}{n}$	Average of all values
Median	Middle value (after sorting)	Less affected by outliers
Mode	Most frequent value	Commonly occurring value

```
df[['Age','Salary']].mean()
df[['Age','Salary']].median()
df[['Age','Salary']].mode()
```

4. Measures of Dispersion

Metric	Formula	Description
Range	Max − Min	Spread between highest and lowest
Variance	$\frac{\Sigma(x-\bar{x})^2}{(n-1)}$	Average squared deviation from mean
Std. Deviation	$\sqrt{\text{Variance}}$	Typical distance from the mean
IQR	Q3 − Q1	Middle 50% spread (robust to outliers)

```
df[['Age','Salary']].std()
df[['Age','Salary']].var()
```

5. Measures of Position

Metric	Description
Minimum	Smallest value
Q1 (25th %)	25% of data is below this
Median (50th %)	Middle value
Q3 (75th %)	75% of data is below this
Maximum	Largest value
Percentile	Value below which a given % of data falls

```
df['Salary'].min()
df['Salary'].quantile(0.25)
df['Salary'].quantile(0.75)
df['Salary'].max()
```

6. Measures of Shape

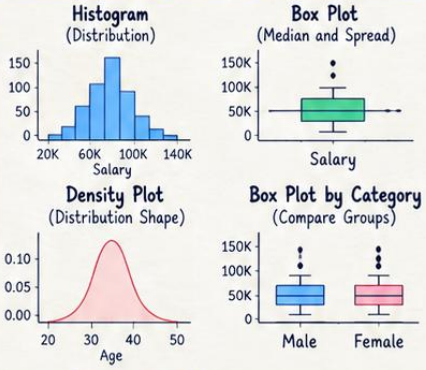
Metric	Description
Skewness	Measures asymmetry of the distribution • > 0 : right skewed • < 0 : left skewed • = 0 : symmetric
Kurtosis	Measures the heaviness of tails • > 0 : heavier tails (more outliers) • < 0 : lighter tails

```
df['Salary'].skew()
df['Salary'].kurt()
```

7. Example using a Dataset

Statistic	Age	Salary	Experience
Count	1000	1000	1000
Mean	29.5	65000	5.2
Median	28.0	62000	4.0
Mode	25.0	60000	3.0
Std. Dev.	4.8	15000	2.9
Min	20	30000	0
Q1 (25%)	26	50000	3
Q3 (75%)	33	78000	7
Max	45	120000	15

8. Visualizing Descriptive Statistics



9. Key Takeaways

- ✅ Use descriptive statistics to get a quick understanding of your data.
- ✅ Check central tendency, spread and distribution shape.
- ✅ Always look for outliers and unusual values.
- ✅ Compare statistics across different groups or categories.
- ✅ Use visualizations along with statistics for better insights.
- ✅ Helps in data cleaning, feature engineering and further analysis.

DataXOdyssey

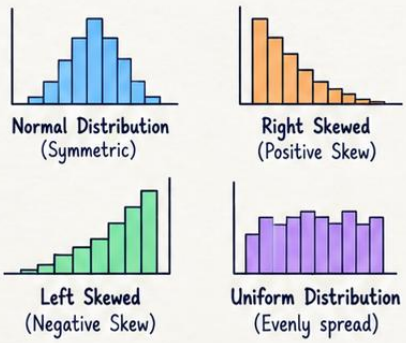
Distribution Analysis

Understanding the shape, spread and patterns in your data

1. What is Distribution Analysis?

Distribution analysis helps us understand how values are spread in a variable, identify its shape, central tendency, variability and any unusual patterns like skewness or outliers.

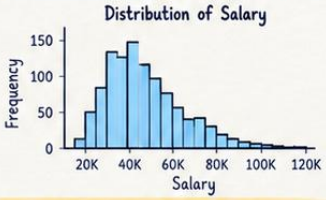
2. Common Distribution Shapes



3. Histogram

Shows the frequency of values in different ranges (bins).

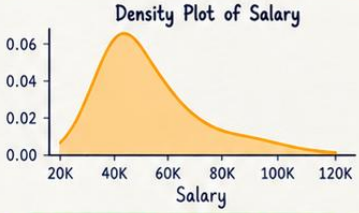
```
import seaborn as sns
sns.histplot(df['Salary'],
             bins=20,
             color='skyblue',
             edgecolor='black')
```



4. Density Plot

Shows a smooth curve to visualize the distribution shape.

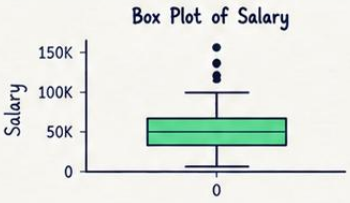
```
sns.kdeplot(df['Salary'],
            fill=True,
            color='orange')
```



5. Box Plot

Shows the median, spread and outliers in the distribution.

```
sns.boxplot(y=df['Salary'],
            color='lightgreen')
```



6. Understanding Skewness

Skewness measures the asymmetry of a distribution.

Skewness Value	Interpretation
= 0	Symmetric distribution
> 0	Right skewed (long tail on right)
< 0	Left skewed (long tail on left)

```
df['Salary'].skew()
# > 0 : right skewed
# < 0 : left skewed
# ~ 0 : symmetric
```

7. Understanding Kurtosis

Kurtosis measures the heaviness of the tails compared to a normal distribution.

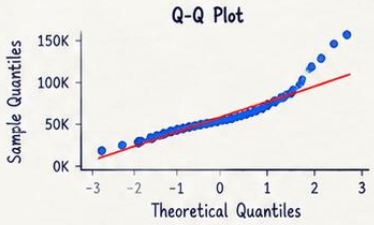
Kurtosis Value	Interpretation
= 3	Normal distribution (mesokurtic)
> 3	Heavy tails (leptokurtic)
< 3	Light tails (platykurtic)

```
df['Salary'].kurt()
# > 3 : heavy tails (more outliers)
# < 3 : light tails (less outliers)
# ~ 3 : similar to normal distribution
```

8. QQ Plot (Normality Check)

Compares the data distribution with a normal distribution.

```
import scipy.stats as stats
stats.probplot(df['Salary'],
               dist='norm', plot=plt)
```



9. Key Takeaways

- ✅ Use histograms and density plots to understand the overall shape.
- ✅ Check for skewness and kurtosis to identify asymmetry and tail behavior.
- ✅ Look for outliers using box plots.
- ✅ Compare with a normal distribution using QQ plots.
- ✅ Understand the distribution before applying transformations or building models.

DataXOdyssey

Outlier Detection & Analysis

Identify, understand and handle unusual values in your data

1. What are Outliers?

Outliers are data points that are significantly different from the rest of the data. They can occur due to data entry errors, natural variation or rare but important events.

2. Why Do Outliers Matter?

- ✅ Can affect summary statistics (mean, standard deviation)
- ✅ May indicate data quality issues
- ✅ Can provide valuable insights (rare events)
- ✅ Important to analyze before removing

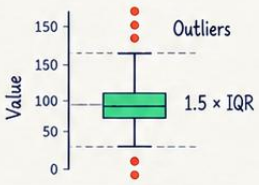
3. Visualizing Outliers



4. IQR Method (Most Common)

Values outside the range [Q1 - 1.5×IQR, Q3 + 1.5×IQR] are considered outliers.

IQR = Q3 - Q1
Lower bound = Q1 - 1.5 × IQR
Upper bound = Q3 + 1.5 × IQR




```
Q1 = df['Salary'].quantile(0.25)
Q3 = df['Salary'].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR
outliers_iqr = df[(df['Salary'] < lower) | (df['Salary'] > upper)]
```

5. Z-Score Method

Values with a Z-score greater than 3 or less than -3 are often considered outliers.

$$Z = \frac{X - \mu}{\sigma}$$

```
from scipy import stats
df['z_score'] = stats.zscore(df['Salary'])
outliers_z = df[abs(df['z_score']) > 3]
```

Value	Z-Score	Outlier?
50000	-0.5	No
75000	0.2	No
120000	2.8	No
250000	4.1	Yes

6. Other Methods

- **Modified Z-Score** (robust to outliers)
- **Percentile Method** (e.g. 1st and 99th percentiles)
- **Domain Knowledge** (business-specific thresholds)
- **Visual Inspection** using scatter plots, histograms and box plots

7. Example Using Dataset

ID	Age	Salary	Experience
1	25	50000	2
2	30	62000	5
3	28	58000	4
4	35	120000	10
5	40	250000	20

```
import pandas as pd
Q1 = df['Salary'].quantile(0.25)
Q3 = df['Salary'].quantile(0.75)
IQR = Q3 - Q1
outliers = df[(df['Salary'] < Q1 - 1.5*IQR) |
              (df['Salary'] > Q3 + 1.5*IQR)]
print(outliers)
```

8. Handling Outliers

1. Verify the outliers (Check for data entry errors)
2. Keep them if they are valid and meaningful
3. Transform the data (e.g. log transformation)
4. Cap/Winsorize the values (e.g. 1st and 99th percentile)
5. Remove outliers only if they are errors and not meaningful

9. Key Takeaways

- ✅ Use multiple methods to detect outliers
- ✅ Always investigate the cause before removing
- ✅ Outliers can be errors or real but rare events
- ✅ Visualizations help in quick identification
- ✅ Choose the right handling method based on the context and business goal

DataXOdyssey

Categorical Data Analysis

Understanding and analyzing categorical variables in your data

1. What are Categorical Variables?

Categorical variables represent groups or categories rather than numeric values. They are used to classify data into meaningful categories for analysis.

2. Types of Categorical Data

Nominal
Categories with no natural order (e.g. City, Gender, Department)

Ordinal
Categories with a meaningful order (e.g. Low, Medium, High)

3. View Categorical Columns

```
df.select_dtypes(include='object')
```

Column	Data Type
Department	object
Gender	object
City	object
Education_Level	object
Job_Role	object

4. Frequency Count

Count the number of records in each category.

```
df['Department'].value_counts()
```

Department	Count
Sales	250
HR	180
Finance	160
IT	220
Operations	190

5. Relative Frequency (Percentage)

Understand the proportion of each category.

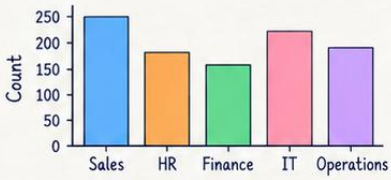
```
df['Department'].value_counts(
    normalize=True) * 100
```

Department	Percentage
Sales	25.0%
HR	18.0%
Finance	16.0%
IT	22.0%
Operations	19.0%

6. Visualizing Categorical Data

Bar Chart (Count)

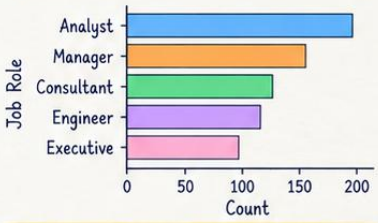
```
import seaborn as sns
sns.countplot(x='Department',
    data=df, order=df['Department']
    .value_counts().index)
```



7. Horizontal Bar Chart

Useful for long category names.

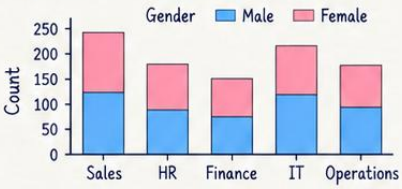
```
sns.countplot(y='Job_Role',
    data=df, order=df['Job_Role']
    .value_counts().index)
```



8. Stacked Bar Chart

Compare categories across a group.

```
pd.crosstab(df['Department'],
  df['Gender']).plot(kind='bar',
  stacked=True, figsize=(6,4))
```



9. Cross Tabulation

Analyze relationship between two categorical variables.

```
pd.crosstab(df['Department'],
  df['Gender'])
```

Department	Male	Female	Total
Sales	140	110	250
HR	90	90	180
Finance	100	60	160
IT	140	80	220
Operations	110	80	190

10. Cardinality & Rare Categories

- Check number of unique categories.
- Identify rare categories (very low count).
- Consider grouping rare categories for better analysis.

```
df['City'].nunique()   # total unique
df['City'].value_counts().tail()   # rare categories
```

11. Handling Categorical Data

1. Standardize category names (e.g. 'usa', 'USA', 'Usa').
2. Handle missing values.
3. Group rare categories.
4. Create meaningful categories (e.g. age groups, income bands).

12. Key Takeaways

- ✅ Understand the type of categorical data (nominal or ordinal).
- ✅ Use frequency and percentage to understand the distribution.
- ✅ Visualize using bar charts and stacked charts.
- ✅ Look for rare categories and group them.
- ✅ Use cross tabulation to analyze relationships between categorical variables.

DataXOdyssey

Bivariate Analysis

Analyzing the relationship between two variables

1. What is Bivariate Analysis?

Bivariate analysis studies the relationship between two variables to understand patterns, trends and associations in the data.

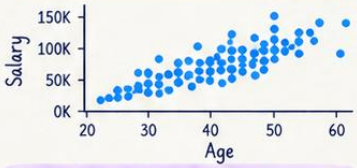
2. Types of Variable Pairs

Variable Pair	Example
Numeric - Numeric	Age vs Salary
Categorical - Numeric	Department vs Salary
Categorical - Categorical	Gender vs Department
Date/Time - Numeric	Month vs Sales

3. Numeric vs Numeric (Scatter Plot)

Shows the relationship between two numeric variables.

```
import seaborn as sns
sns.scatterplot(x=df['Age'],
                y=df['Salary'])
```



4. Correlation Analysis

Measures the strength and direction of the linear relationship between two numeric variables.

Correlation Value	Interpretation
+1	Perfect positive correlation
0	No linear correlation
-1	Perfect negative correlation

```
df[['Age','Salary']].corr()
```

5. Visualizing Correlation

Correlation heatmap for multiple numeric variables.

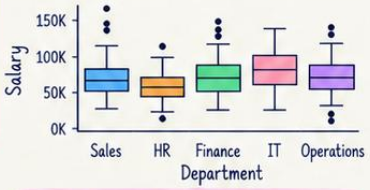
```
import seaborn as sns
sns.heatmap(df.corr(numeric_only=True),
            annot=True, cmap='coolwarm')
```

	Age	Salary	Experience	Performance
Age	1.00	0.78	0.86	0.32
Salary	0.78	1.00	0.76	0.40
Experience	0.86	0.76	1.00	0.28
Performance	0.32	0.40	0.28	1.00

6. Categorical vs Numeric (Box Plot)

Compares the distribution of a numeric variable across categories.

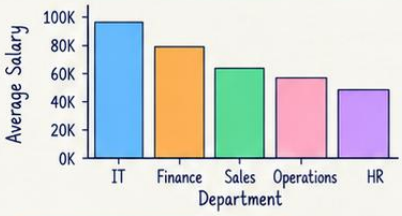
```
sns.boxplot(x=df['Department'],
            y=df['Salary'])
```



7. Categorical vs Numeric (Bar Chart with Aggregation)

Compares the average (or sum) of a numeric variable across categories.

```
df.groupby('Department')['Salary']
  .mean().sort_values(ascending=False)
```



8. Categorical vs Categorical (Cross Tabulation)

Analyzes the relationship between two categorical variables.

```
pd.crosstab(df['Department'],
            df['Gender'])
```

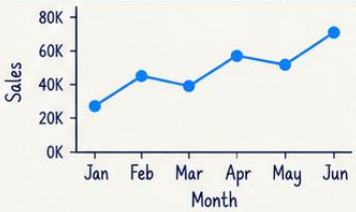
Department	Male	Female	Total
Sales	140	110	250
HR	60	120	180
Finance	100	60	160
IT	140	80	220
Operations	110	80	190

```
pd.crosstab(df['Department'], df['Gender'],
            normalize='index') * 100  # percentage
```

9. Trends Over Time (Numeric vs Date)

Analyze how a numeric variable changes over time.

```
df.groupby('Month')['Sales'].sum()
  .plot(kind='line', marker='o')
```



10. Key Takeaways

- ✅ Use the right visualization for the type of variable pair.
- ✅ Look for patterns, trends and outliers.
- ✅ Correlation shows association, not causation.
- ✅ Use aggregation for categorical vs numeric.
- ✅ Cross tabulation helps analyze relationships between categories.
- ✅ Bivariate analysis is useful for initial insights and feature selection in modeling.

DataXOdyssey

Multivariate Analysis

Understanding relationships among multiple variables

1. What is Multivariate Analysis?

Multivariate analysis looks at multiple variables together to understand relationships, patterns, interactions and hidden insights in the data.

2. Why is it Important?

- ✅ Find complex relationships
- ✅ Identify key drivers
- ✅ Understand interactions between variables
- ✅ Detect patterns and segments
- ✅ Helps in feature selection and model building

3. Pair Plot (Multiple Variables)

Shows distribution and pairwise relationships between multiple numeric variables.

```
import seaborn as sns
sns.pairplot(df[['Age','Salary',
   'Experience','Performance']])
```



4. Correlation Matrix & Heatmap

Shows the strength and direction of relationship between multiple numeric variables.

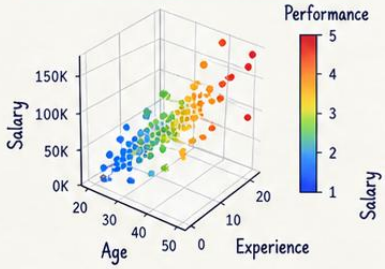
```
import seaborn as sns
sns.heatmap(df.corr(numeric_only=True),
   annot=True, cmap='coolwarm')
```



5. 3D Scatter Plot

Visualizes the relationship between three numeric variables.

```
import plotly.express as px
px.scatter_3d(df, x='Age',y='Salary',
  z='Experience',color='Performance')
```



6. Grouped Analysis (Multiple Variables)

Compare a numeric variable across two or more categorical variables.

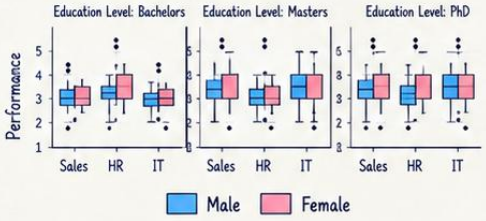
```
sns.boxplot(x='Department',
y='Salary', hue='Gender', data=df)
```



7. Feature Relationship (Multiple Categories)

Analyze how a numeric variable changes across combinations of categories.

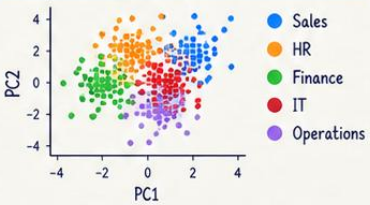
```
sns.catplot(x='Department', y='Performance',
   hue='Gender', col='Education_Level',
   kind='box', data=df)
```



8. Dimensionality Reduction (PCA)

Reduces multiple correlated variables into few principal components for easier visualization.

```
from sklearn.decomposition import PCA
pca = PCA(n_components=2)
components = pca.fit_transform(df[numeric_cols])
df_pca = pd.DataFrame(components,
   columns=['PC1','PC2'])
```



9. Key Takeaways

- ✅ Use pair plots and heatmaps to understand relationships.
- ✅ Analyze interactions between multiple variables.
- ✅ Compare groups using box plots and grouped charts.
- ✅ Use dimensionality reduction (e.g. PCA) for high-dimensional data.
- ✅ Helps in feature selection, segmentation and building better models.

DataXOdyssey

Correlation & Statistical Relationships

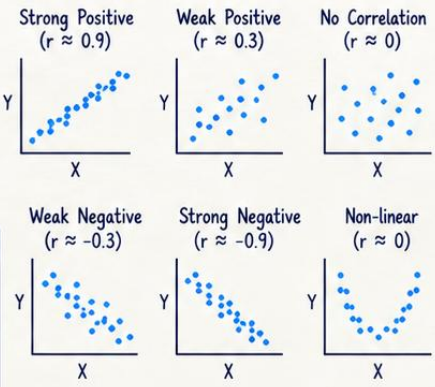
Understanding the strength, direction and significance of relationships in your data

1. What is Correlation?

Correlation measures the strength and direction of the linear relationship between two numeric variables. The value ranges from -1 to +1.

Correlation Value	Interpretation
+1	Perfect positive linear relationship
0	No linear relationship
-1	Perfect negative linear relationship

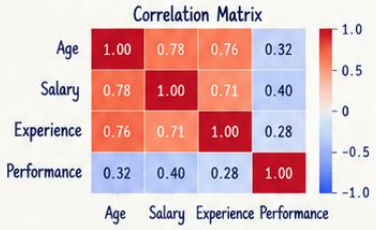
2. Types of Relationships



3. Correlation Matrix

Shows the correlation between multiple numeric variables.

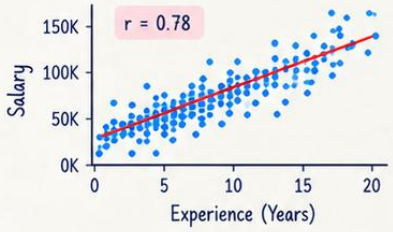
```
import seaborn as sns
plt.figure(figsize=(6,4))
sns.heatmap(df.select_dtypes
  (include='number').corr(),
  annot=True, cmap='coolwarm',
  vmin=-1, vmax=1)
```



4. Scatter Plot with Correlation

Visualizes the relationship between two numeric variables.

```
sns.scatterplot(x=df['Experience'],
                y=df['Salary'])
```



5. Correlation with p-value

Check if the correlation is statistically significant.

```
from scipy.stats import pearsonr
corr, p_value = pearsonr(
df['Experience'], df['Salary'])
print(corr, p_value)
```

Metric	Value
Correlation (r)	0.78
p-value	0.0001

- r = 0.78 indicates strong positive correlation.
- p-value < 0.05 indicates the relationship is statistically significant.
- p-value ≥ 0.05 indicates no significant relationship.

6. Spearman Correlation (For Non-Linear / Ordinal Data)

Measures the strength of monotonic relationship (works with ordinal or non-linear data).

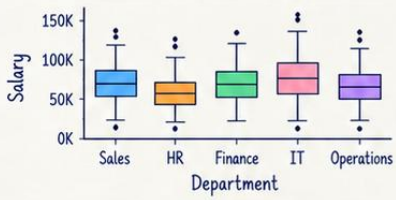
```
from scipy.stats import spearmanr
corr, p_value = spearmanr(
    df['Satisfaction_Rating'],
    df['Performance_Score'])
```

Spearman r	Interpretation
+1	Strong positive monotonic
0	No monotonic relationship
-1	Strong negative monotonic

7. Correlation with Categorical Variable

Compare a numeric variable across categories using box plot.

```
sns.boxplot(x=df['Department'],
            y=df['Salary'])
```



8. Other Statistical Relationships

Test / Method	Use Case	Function (Python)
Pearson Correlation	Linear relationship (both numeric)	scipy.stats .pearsonr()
Spearman Correlation	Monotonic / ordinal relationship	scipy.stats .spearmanr()
ANOVA	Compare mean across 3+ groups	scipy.stats .f_oneway()
T-test	Compare mean between 2 groups	scipy.stats .ttest_ind()
Chi-Square Test	Association between categorical variables	scipy.stats .chi2_contingency()

9. Key Takeaways

- Use correlation to understand strength and direction of relationships.
- Check p-values to confirm statistical significance.
- Use appropriate methods (Pearson, Spearman, ANOVA, T-test) based on data type and question.
- Visualize relationships using scatter plots, heatmaps and box plots.
- Remember: correlation does not imply causation.

DataXOdyssey

Time-Based EDA

Analyzing trends, seasonality and patterns in time series data

1. What is Time-Based EDA?

Time-based EDA analyzes data with a time component to understand trends, seasonality, patterns and anomalies over time.

2. Converting to Date/Time

Ensure the date column is in datetime format.

```
df['Date'] = pd.to_datetime(df['Date'])
df = df.sort_values('Date')
df.set_index('Date', inplace=True)
```

Date	Sales	Product
2023-01-01	1200	A
2023-01-02	900	B
2023-01-03	1500	A
2023-01-04	1100	B
2023-01-05	1700	A

3. Time Components

Extract useful time-based features for analysis.

```
df['Year'] = df.index.year
df['Month'] = df.index.month
df['Day'] = df.index.day
df['DayOfWeek'] = df.index.day_name()
df['Quarter'] = df.index.quarter
```

Date	Year	Month	Day	DayOfWeek	Quarter
2023-01-01	2023	1	1	Sunday	1
2023-01-02	2023	1	2	Monday	1
2023-01-03	2023	1	3	Tuesday	1
2023-04-15	2023	4	15	Saturday	2
2023-10-20	2023	10	20	Friday	4

4. Overall Trend Analysis

Visualize how the metric changes over time.

```
df['Sales'].plot(kind='line',
  figsize=(6,4), title='Sales Over Time')
```



5. Seasonality Analysis

Identify repeating patterns by month, quarter or day of week.

```
df.groupby('Month')['Sales'].mean().
  plot(kind='bar', title='Average Sales
  by Month', figsize=(6,4))
```



6. Day of Week Analysis

Check which day of the week performs better.

```
df.groupby('DayOfWeek')['Sales'].mean().
  reindex(['Monday','Tuesday','Wednesday',
  'Thursday','Friday','Saturday','Sunday']).
  plot(kind='bar', figsize=(6,4))
```



7. Rolling Statistics

Smooth the data to understand underlying trend.

```
df['Sales_MA7'] = df['Sales'].rolling(7).mean()
df[['Sales','Sales_MA7']].plot(figsize=(6,4))
```



8. Resampling & Aggregation

Aggregate data to different time frequencies (day, week, month).

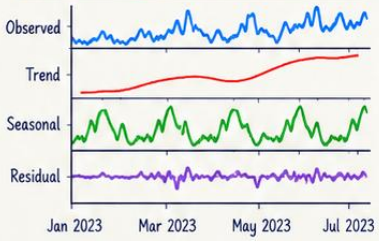
```
monthly_sales = df['Sales'].resample('M').sum()
weekly_sales = df['Sales'].resample('W').sum()
quarterly_sales = df['Sales'].resample('Q').sum()
```



9. Anomaly Detection

Identify unusual spikes or drops in the data.

```
from statsmodels.tsa.seasonal import
    seasonal_decompose
result = seasonal_decompose(df['Sales'],
        period=30)
result.plot(figsize=(6,4))
```



10. Key Takeaways

- ✅ Always convert date column to datetime.
- ✅ Analyze trends, seasonality and patterns.
- ✅ Use aggregation and resampling to understand the data at different levels.
- ✅ Look for unusual spikes or drops.
- ✅ Time-based EDA helps in demand forecasting, capacity planning and decision making.
- ✅ Combine time-based insights with other variables for deeper analysis.

DataXOdyssey

Grouped & Segment Analysis

Analyzing data across groups and segments to uncover meaningful insights

1. What is Grouped & Segment Analysis?

It involves splitting data into different groups or segments to compare behavior, identify patterns and uncover key insights.

2. Grouping with Pandas

Use groupby() to calculate aggregations for each group.

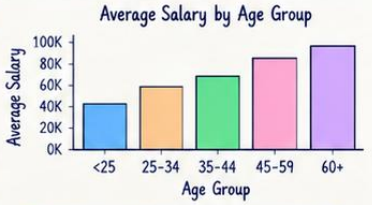
```
df.groupby('Department')['Salary'].agg(
 ['count', 'mean', 'median', 'min', 'max']
)
```

Department	Count	Mean	Median	Min	Max
Sales	250	72000	70000	40000	120000
HR	180	65000	62000	38000	110000
Finance	160	78000	75000	45000	130000
IT	220	82000	80000	50000	150000
Operations	190	68000	67000	39000	125000

3. Segmenting Continuous Variables

Create bins to segment numeric variables (e.g. age group, income band).

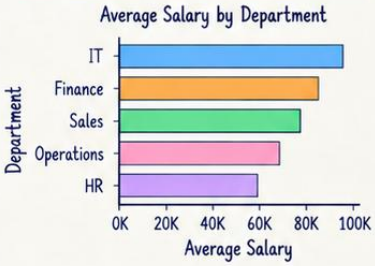
```
df['Age_Group'] = pd.cut(df['Age'],
 bins=[0,25,35,45,60,100],
labels=['<25','25-34','35-44','45-59','60+'])
```



4. Comparing Groups

Compare key metrics across segments.

```
df.groupby('Department')['Salary']
 .mean().sort_values(ascending=False)
```



5. Multiple Grouping (Segment Analysis)

Analyze using multiple grouping variables.

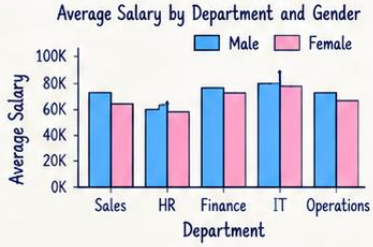
```
df.groupby(['Department','Gender'])['Salary']
   .mean().reset_index()
```

Department	Gender	Count	Average Salary
Sales	Male	140	74000
Sales	Female	110	69000
HR	Male	90	66000
HR	Female	90	64000
Finance	Male	100	80000
Finance	Female	60	74500

6. Visualizing Segments

Use grouped bar charts to compare segments.

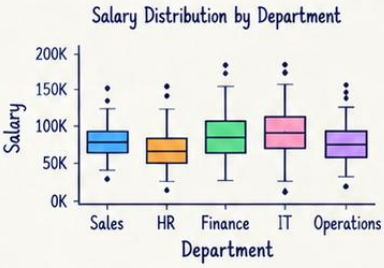
```
sns.barplot(x='Department', y='Salary',
hue='Gender', data=df, estimator='mean')
```



7. Distribution Across Segments

Compare distribution using box plots.

```
sns.boxplot(x='Department', y='Salary',
 data=df)
```



8. Segment Size and Contribution

Check the size and contribution of each segment to the total.

```
segment_summary = df.groupby('Department').
  agg(Count=('EmployeeID','count'),
      Total_Salary=('Salary','sum'))
segment_summary['Salary_Contribution_%'] =
  (segment_summary['Total_Salary'] /
  segment_summary['Total_Salary'].sum()) * 100
```

Department	Count	Total Salary	Contribution %
IT	220	18,040,000	26.8%
Finance	160	12,480,000	18.6%
Sales	250	18,000,000	26.7%
Operations	190	12,920,000	19.2%
HR	180	5,760,000	8.6%

9. Key Takeaways

- ✅ Group data to compare performance across segments.
- ✅ Use multiple grouping variables for deeper analysis.
- ✅ Create meaningful segments for continuous variables.
- ✅ Visualize using bar charts, box plots and grouped charts.
- ✅ Analyze segment size and contribution to understand business impact.
- ✅ Helps in identifying high/low performing groups, target areas and opportunities for action.

DataXOdyssey

EDA Using Python & Pandas

Step-by-step exploration of data using Python and Pandas

1. Load Data & Basic Setup

Import required libraries and load the dataset.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
# Load data
df = pd.read_csv('data.csv')
df.head()
```

Date	Product	Sales	Region
2023-01-01	A	1200	North
2023-01-02	B	900	South
2023-01-03	A	1500	East
2023-01-04	C	1100	West
2023-01-05	B	1700	North

2. Understand the Data

Check structure, data types, missing values and summary.

```
df.shape
df.info()
df.describe()
df.isnull().sum()
df.nunique()
```

df.info() (Sample Output)

```
RangeIndex: 1000 entries, 0 to 999
Data columns (total 5 columns):
 #   Column   Non-Null Count  Dtype
 0   Date     1000 non-null   object
 1   Product  1000 non-null   object
 2   Sales    1000 non-null   int64
 3   Region   1000 non-null   object
 4   Profit    950 non-null   float64
dtypes: float64(1), int64(1), object(3)
memory usage: 39.2+ KB
```

3. Clean the Data

Handle missing values, duplicates and incorrect formats.

```
# Convert date column
df['Date'] = pd.to_datetime(df['Date'])

# Handle missing values
df['Profit'] = df['Profit'].fillna(
    df['Profit'].median())

# Remove duplicates
df = df.drop_duplicates()


# Check data types
df.dtypes
```

4. Explore Numeric Variables

Check distribution and summary statistics.

```
df[['Sales', 'Profit']].describe()
df[['Sales', 'Profit']].hist(
figsize=(8,3), bins=20)
plt.show()
```

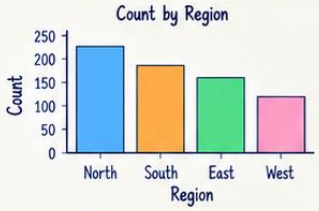


Metric	Sales	Profit
Count	1000	950
Mean	1050.20	280.45
Median	980.00	260.00
Std	420.30	120.50
Min	200.00	-50.00
Max	2100.00	650.00

5. Explore Categorical Variables

Check unique values, frequency and distribution.

```
df['Region'].value_counts()
df['Product'].value_counts()
sns.countplot(x='Region', data=df,
    order=df['Region'].value_counts()
    .index)
plt.show()
```

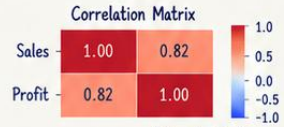


6. Bivariate Analysis

Explore relationships between variables.

```
# Scatter plot
sns.scatterplot(x='Sales', y='Profit',
    data=df)
plt.show()
# Correlation
sns.heatmap(df[['Sales','Profit'],
    annot=True, cmap='coolwarm')
plt.show()
```





7. Time-Based Analysis

Analyze trends and seasonality using date column.

```
df['Month'] = df['Date'].dt.month
df['Year'] = df['Date'].dt.year
df.groupby('Month')['Sales']
  .sum().plot(kind='bar',figsize=(6,3))
plt.xlabel('Month')
plt.ylabel('Total Sales')
plt.show()
```



8. Grouped & Segment Analysis

Use groupby to compare performance across segments.

```
# Average sales by region
df.groupby('Region')['Sales']
  .mean().sort_values(ascending=False)

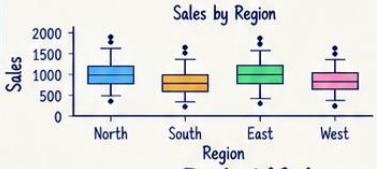
# Multiple aggregation
df.groupby('Product').agg({
    'Sales':['sum','mean','count'],
    'Profit':'mean'})
```

Product	Sales (sum)	Sales (mean)	Count	Profit (mean)
A	420,000	1,400	300	350.2
B	310,000	1,033	300	260.5
C	270,000	900	300	230.1

9. Outlier Detection

Use box plots and IQR method.

```
sns.boxplot(x='Region', y='Sales',
    data=df)
plt.show()
```



10. Key Takeaways

- Understand data structure and quality.
- Analyze distributions and relationships.
- Use visualizations to find patterns and outliers.
- Compare groups and segments.
- Explore time-based trends and seasonality.
- Use insights to support feature selection and business decisions.

DataXOdyssey

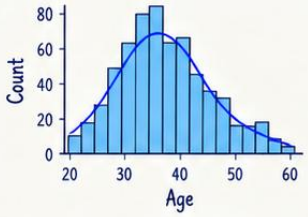
EDA Visualization Toolkit

Essential plots to explore, understand and communicate your data

1. Histogram (Distribution)

Shows the distribution of a numeric variable.

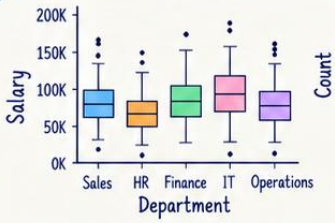
```
sns.histplot(df['Age'],
  bins=30, kde=True, color='skyblue')
plt.xlabel('Age')
plt.ylabel('Count')
```



2. Box Plot (Spread & Outliers)

Shows median, spread and outliers in the data.

```
sns.boxplot(x='Department',
  y='Salary', data=df,
  palette='Set2')
```



3. Bar Plot (Categorical)

Compares counts or aggregated values across categories.

```
sns.countplot(x='Department',
  data=df, palette='Pastel1')
```



4. Pie Chart (Share of Total)

Shows the proportion of each category.

```
df['Region'].value_counts().plot(
  kind='pie', autopct='%1.1f%%',
  cmap='Pastel1', startangle=90)
plt.ylabel('')
```



5. Line Plot (Trend Over Time)

Shows how a metric changes over time.

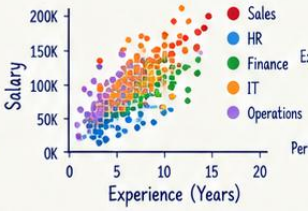
```
df['Date'] = pd.to_datetime(df['Date'])
sns.lineplot(x='Date', y='Sales',
  data=df, color='blue')
```



6. Scatter Plot (Relationship)

Shows the relationship between two numeric variables.

```
sns.scatterplot(x='Experience',
  y='Salary', data=df,
  hue='Department', palette='Set1')
```



7. Heatmap (Correlation)

Shows correlation between numeric variables.

```
sns.heatmap(df.corr(numeric_only=True),
  annot=True, cmap='coolwarm',
  fmt='.2f')
```



8. Pair Plot (Multiple Variables)

Shows distribution and pairwise relationships.

```
sns.pairplot(df[['Age','Salary',
  'Experience','Performance']],
  hue='Department',
  palette='Set2')
```



9. Count Plot with Percentage

Shows count and percentage for categories.

```
ax = sns.countplot(x='Department',
  data=df, palette='Pastel1')
for p in ax.patches:
  ax.annotate(f'{p.get_height()}',
  (p.get_x()+p.get_width()/2, p.get_height()+5),
  ha='center', va='bottom')
```



10. Violin Plot (Distribution)

Shows distribution and density across categories.

```
sns.violinplot(x='Department',
  y='Salary', data=df,
  palette='Set2')
```



11. Stacked Bar Chart

Compares group composition across categories.

```
pd.crosstab(df['Department'],
  df['Gender']).plot(kind='bar',
  stacked=True, color=['skyblue','salmon'])
```



12. Key Takeaways

- Use the right chart for the right type of data.
- Visualize distribution, trends, relationships and composition.
- Combine multiple plots for a complete understanding.
- Customize plots with labels, colors and titles.
- Use visualization to identify patterns, outliers and key insights.
- Clear visuals help in better decision making and communication.

DataXOdyssey

End-to-End EDA Case Study

Complete exploratory data analysis workflow using a real-world dataset (Sales)

1. Business Understanding

Understand the problem, data and key questions to explore.

Dataset: Retail Sales
Columns: Order Date, Region, Category, Product, Sales, Quantity, Discount, Profit, Customer Segment

Key Questions:

- What are the overall sales and profit trends?
- Which products and regions perform best?
- Are there seasonal patterns?
- What customer segments drive revenue?
- Identify outliers and unusual patterns.

2. Load Data & Initial Inspection

Import libraries, load the data and check basic information.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv('sales.csv')
df.head()
df.info()
df.describe()
```

Sample Data

Order Date	Region	Category	Sales	Profit
2023-01-05	East	Furniture	1200	200
2023-01-08	West	Technology	900	150
2023-01-12	Central	Office Supplies	450	50
2023-01-15	South	Furniture	700	-40
2023-01-17	East	Technology	1600	320

3. Data Cleaning

Handle missing values, duplicates and incorrect formats.

```
# Check missing values
df.isnull().sum()

# Convert date column
df['Order Date'] = pd.to_datetime
(df['Order Date'])

# Remove duplicates
df = df.drop_duplicates()

# Check data types
df.dtypes
```

Missing Values Before vs After

Column	Before	After
Order Date	0	0
Region	0	0
Category	0	0
Sales	2	0
Profit	2	0

4. Descriptive Statistics

Understand the central tendency, spread and key metrics.

```
df.describe()
df[['Sales','Profit','Quantity',
  'Discount']].describe()
```

Summary Statistics (Selected Columns)

Metric	Sales	Profit	Quantity	Discount
Count	9994	9994	9994	9994
Mean	230.4	28.7	3.8	0.16
Median	120.0	12.0	3.0	0.15
Std	625.3	129.4	2.2	0.21
Min	0.0	-659.0	1.0	0.00
Max	22638.0	8399.98	14.0	0.80

5. Univariate Analysis

Analyze distribution of individual numeric and categorical variables.




```
sns.histplot(df['Sales'],
  bins=30, kde=True)
```




```
sns.countplot(x='Category',
  data=df)
```

6. Bivariate Analysis

Explore relationships between variables.




```
sns.scatterplot(x='Sales',
  y='Profit', data=df)
```



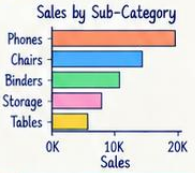

```
sns.boxplot(x='Region',
  y='Sales', data=df)
```

7. Categorical Analysis

Analyze key metrics across categories and segments.




```
df.groupby('Segment')['Sales']
.mean().plot(kind='bar')
```




```
df.groupby('Sub-Category')
['Sales'].sum().sort_values(
ascending=False).plot(kind='barh')
```

8. Time-Based Analysis

Analyze trends and seasonality.




```
df['Month'] = df['Order Date'].dt.month
monthly_sales = df.groupby('Month')['Sales']
  .sum()
monthly_sales.plot(kind='line', marker='o')
```

9. Grouped & Segment Analysis

Compare performance across multiple groups.



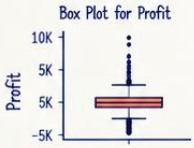

```
df.groupby(['Region','Category']).agg({
'Sales':'sum','Profit':'sum'}).unstack().plot(kind='bar')
```

10. Outlier Detection

Identify unusual values using box plots and IQR.



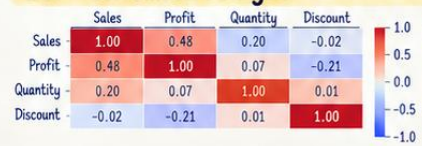

```
sns.boxplot(y='Sales', data=df)
```




```
sns.boxplot(y='Profit', data=df)
```

11. Correlation Analysis

Check correlation between numeric variables.




```
sns.heatmap(df[['Sales','Profit',
 'Quantity','Discount']].corr(),
 annot=True, cmap='coolwarm',
 fmt='.2f')
```

12. Key Insights & Takeaways

- Total sales and profit show an increasing trend over time, with peak in Q4.
- Technology category generates highest sales and profit.
- East and West regions perform better compared to Central and South.
- Consumer segment contributes the highest revenue.
- There are some high-value outliers in sales and profit.
- Sales and profit are moderately correlated (0.48).
- Discount has a negative correlation with profit (-0.21).
- These insights can help in inventory planning, marketing strategy and targeting high-performing segments.

DataXOdyssey